```
import pandas as pd

filename = "student_feedback.csv"

df = pd.read_csv(filename)
df
```


```
{"summary":"{\n  \"name\": \"df\",\n  \"rows\": 1001,\n  \"fields\":
[\n    {\n      \"column\": \"Unnamed: 0\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 289,\n        \"min\": 0,\n
\"max\": 1000,\n        \"num_unique_values\": 1001,\n
\"samples\": [\n          521,\n          941,\n          741\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Student ID\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
289,\n        \"min\": 0,\n        \"max\": 1000,\n
\"num_unique_values\": 1001,\n        \"samples\": [\n          765,\n
677,\n          537\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Well versed with the subject\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1,\n        \"min\": 5,\n
\"max\": 10,\n        \"num_unique_values\": 6,\n        \"samples\":
[\n          5,\n          6,\n          10\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Explains concepts in an
understandable way\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 2,\n        \"min\": 2,\n
\"max\": 10,\n        \"num_unique_values\": 9,\n        \"samples\":
[\n          9,\n          5,\n          3\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Use of presentations\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1,\n        \"min\": 4,\n        \"max\": 8,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          8,\n
4,\n          6\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Degree of difficulty of assignments\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 2,\n        \"min\": 1,\n
\"max\": 10,\n        \"num_unique_values\": 10,\n        \"samples\":
[\n          10,\n          5,\n          2\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Solves doubts willingly\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
2,\n        \"min\": 1,\n        \"max\": 10,\n
\"num_unique_values\": 10,\n        \"samples\": [\n          1,\n
2,\n          3\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Structuring of the course\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 2,\n        \"min\": 1,\n
\"max\": 10,\n        \"num_unique_values\": 10,\n        \"samples\":
```

```
[\n          10,\n          1,\n          3\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Provides support for students
going above and beyond\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 2,\n        \"min\": 1,\n
\"max\": 10,\n        \"num_unique_values\": 10,\n        \"samples\":
[\n          10,\n          2,\n          8\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Course recommendation based on
relevance\",\n      \"properties\": {\n        \"dtype\": \"number\",\
n        \"std\": 2,\n        \"min\": 1,\n        \"max\": 10,\n
\"num_unique_values\": 10,\n        \"samples\": [\n          7,\n
9,\n          10\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\
n}","type":"dataframe","variable_name":"df"}
```


```
df_clean = df.drop(columns=['Unnamed: 0'])
```

```
df_clean
```


```
{"summary":"{\n  \"name\": \"df_clean\",\n  \"rows\": 1001,\n
\"fields\": [\n    {\n      \"column\": \"Student ID\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
289,\n        \"min\": 0,\n        \"max\": 1000,\n
\"num_unique_values\": 1001,\n        \"samples\": [\n          765,\n
677,\n          537\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Well versed with the subject\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 1,\n        \"min\": 5,\n
\"max\": 10,\n        \"num_unique_values\": 6,\n        \"samples\":
[\n          5,\n          6,\n          10\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Explains concepts in an
understandable way\",\n      \"properties\": {\n        \"dtype\":
\"number\",\n        \"std\": 2,\n        \"min\": 2,\n
\"max\": 10,\n        \"num_unique_values\": 9,\n        \"samples\":
[\n          9,\n          5,\n          3\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Use of presentations\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
1,\n        \"min\": 4,\n        \"max\": 8,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          8,\n
4,\n          6\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    },\n    {\n      \"column\":
\"Degree of difficulty of assignments\",\n      \"properties\": {\n
\"dtype\": \"number\",\n        \"std\": 2,\n        \"min\": 1,\n
\"max\": 10,\n        \"num_unique_values\": 10,\n        \"samples\":
[\n          10,\n          5,\n          2\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Solves doubts willingly\",\n
```

\"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 2,\n   \"min\": 1,\n   \"max\": 10,\n   \"num_unique_values\": 10,\n   \"samples\": [\n   1,\n   2,\n   3\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Structuring of the course\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 2,\n   \"min\": 1,\n   \"max\": 10,\n   \"num_unique_values\": 10,\n   \"samples\": [\n   10,\n   1,\n   3\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Provides support for students going above and beyond\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 2,\n   \"min\": 1,\n   \"max\": 10,\n   \"num_unique_values\": 10,\n   \"samples\": [\n   10,\n   2,\n   8\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   },\n   {\n   \"column\": \"Course recommendation based on relevance\",\n   \"properties\": {\n   \"dtype\": \"number\",\n   \"std\": 2,\n   \"min\": 1,\n   \"max\": 10,\n   \"num_unique_values\": 10,\n   \"samples\": [\n   7,\n   9,\n   10\n   ],\n   \"semantic_type\": \"\",\n   \"description\": \"\"\n   }\n   }\n]\n}","type":"dataframe","variable_name":"df_clean"}

```
missing_values = df_clean.isnull().sum()

missing_values
```

```
Student ID                                                0
Well versed with the subject                              0
Explains concepts in an understandable way                0
Use of presentations                                      0
Degree of difficulty of assignments                       0
Solves doubts willingly                                   0
Structuring of the course                                 0
Provides support for students going above and beyond      0
Course recommendation based on relevance                  0
dtype: int64
```

```
avg_scores = df_clean.drop(columns=['Student ID']).mean().sort_values(ascending=False)

avg_scores
```

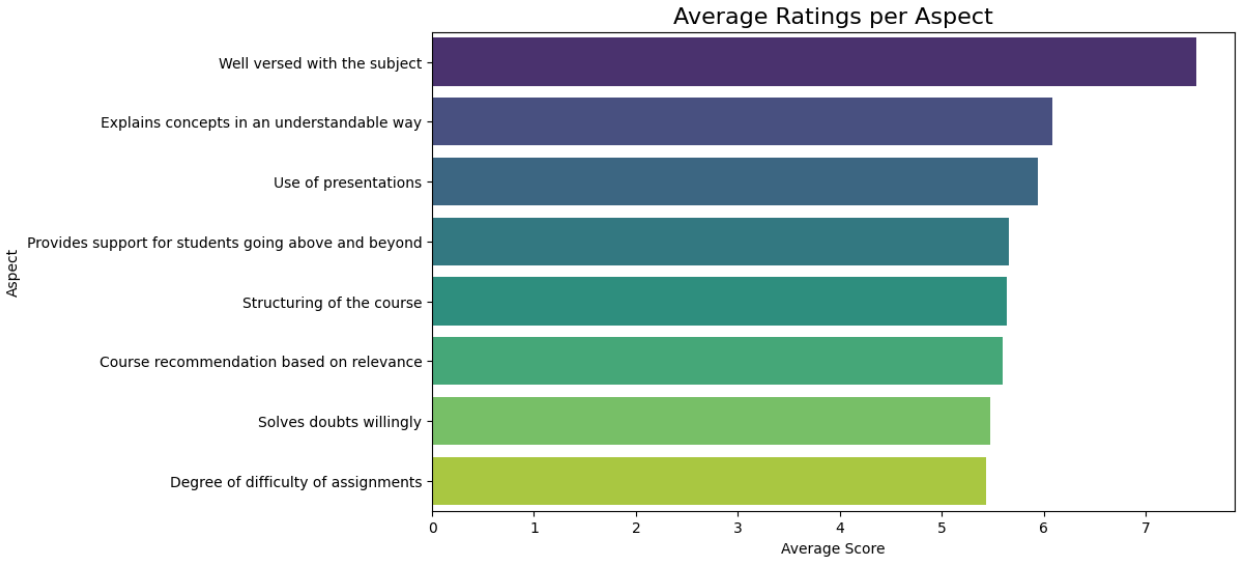
```
Well versed with the subject                              7.497502
Explains concepts in an understandable way                6.081918
Use of presentations                                      5.942058
Provides support for students going above and beyond      5.662338
Structuring of the course                                 5.636364
Course recommendation based on relevance                  5.598402
Solves doubts willingly                                   5.474525
```

```
Degree of difficulty of assignments                    5.430569
dtype: float64
```

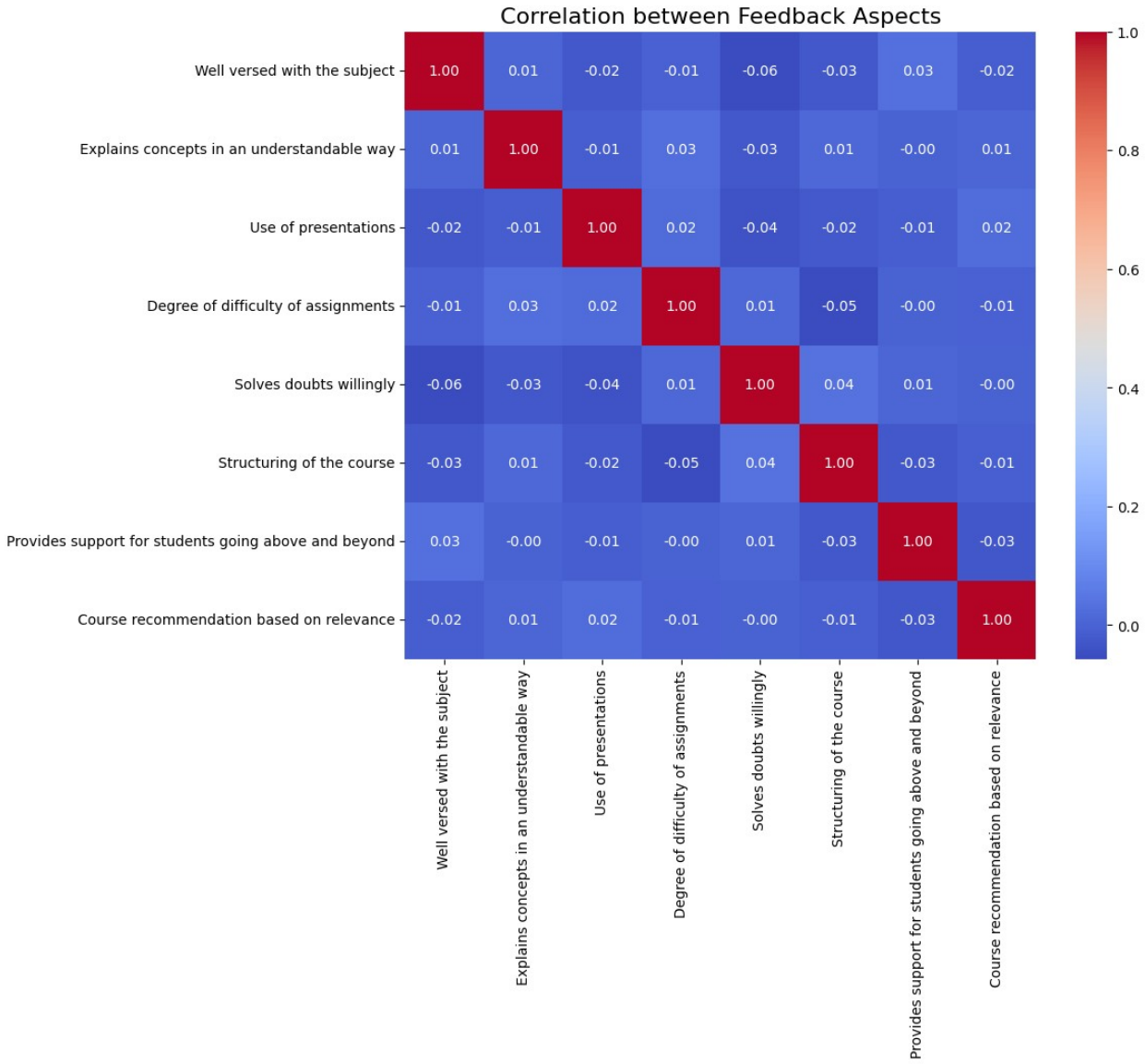
```
overall_satisfaction = df_clean.drop(columns=['Student
ID']).mean(axis=1)

overall_satisfaction
```

```
0       5.000
1       4.875
2       4.375
3       5.875
4       7.500
        ...
996     6.375
997     5.125
998     4.625
999     5.125
1000    4.875
Length: 1001, dtype: float64
```

```
import matplotlib.pyplot as plt
import seaborn as sns
plt.figure(figsize=(10,6))
sns.barplot(
x=avg_scores.values,
y=avg_scores.index,
hue=avg_scores.index,
palette="viridis",
legend=False # hide legend
)
plt.title("Average Ratings per Aspect", fontsize=16)
plt.xlabel("Average Score")
plt.ylabel("Aspect")
plt.show()
```




```
plt.figure(figsize=(10,8))
corr = df_clean.drop(columns=['Student ID']).corr()
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Correlation between Feedback Aspects", fontsize=16)
plt.show()
```

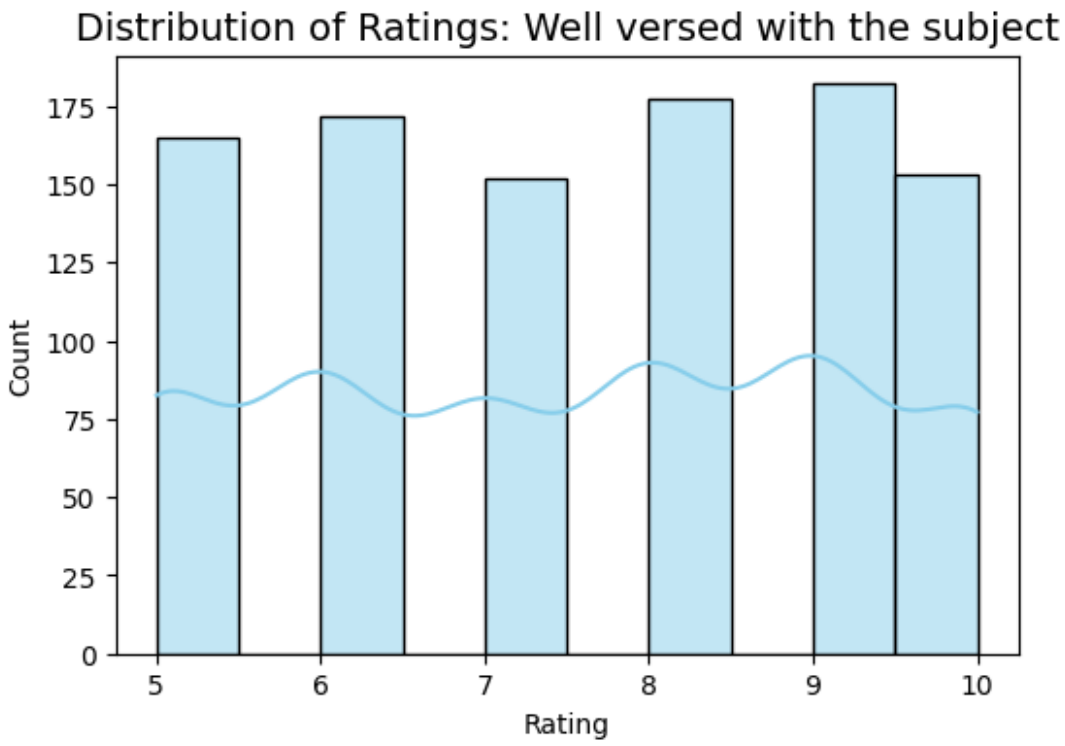


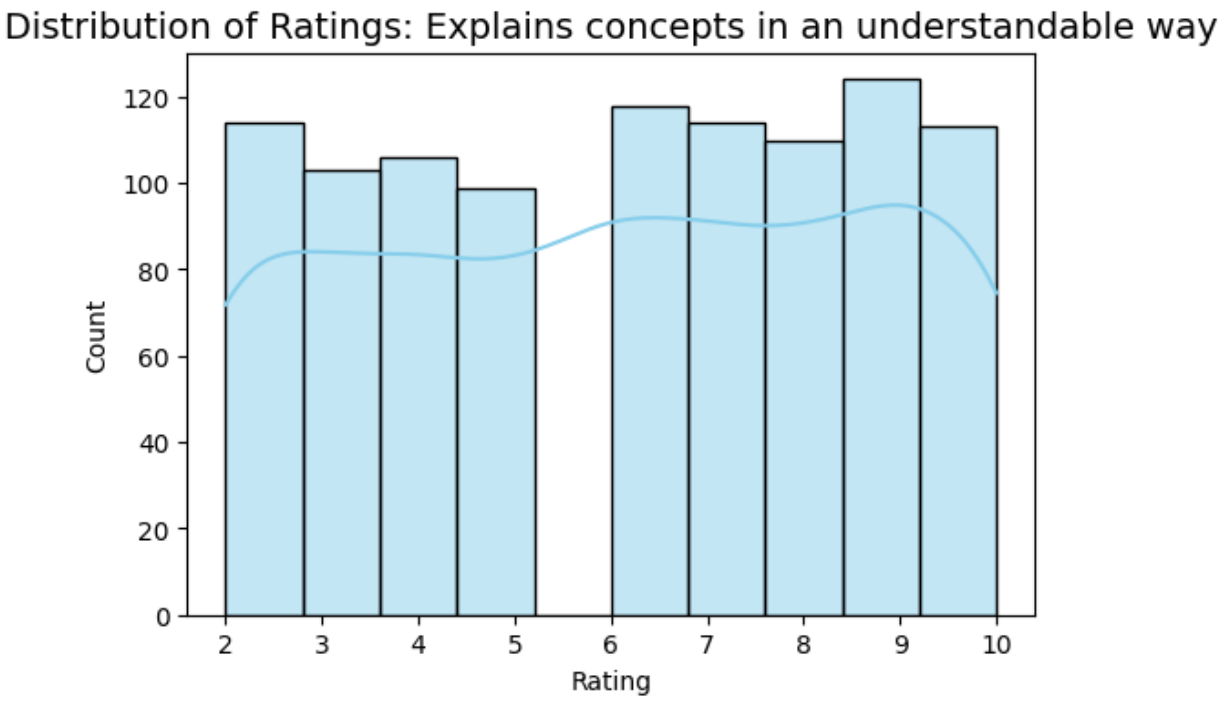

```
missing_values, avg_scores.head(), overall_satisfaction.head()
```

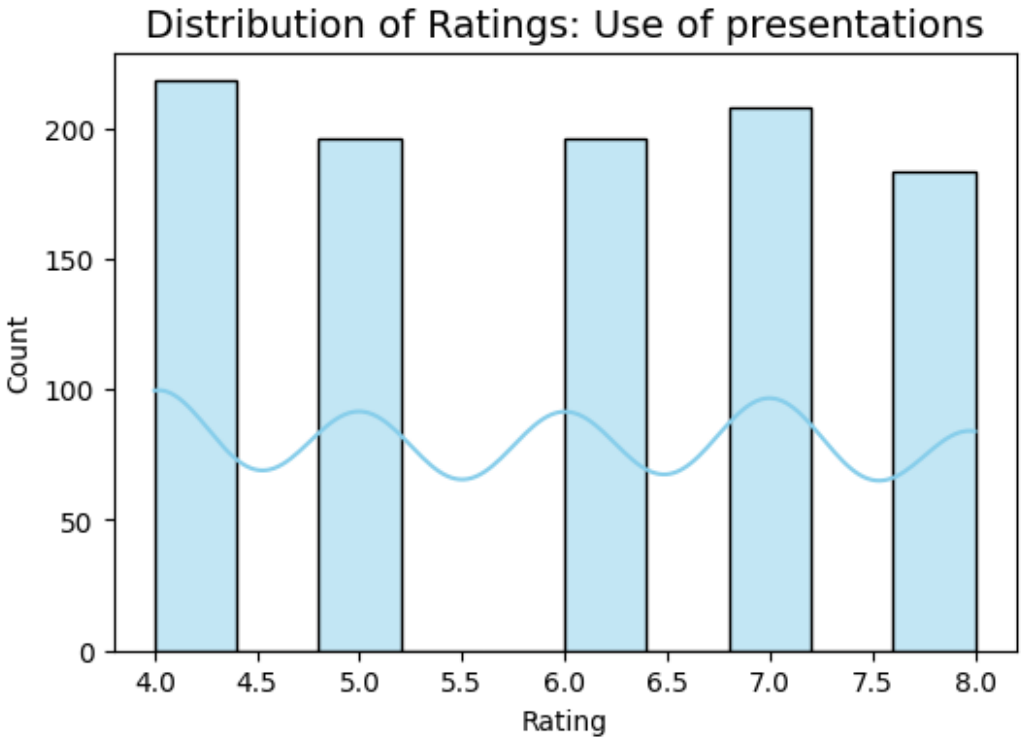
```
(Student ID                                                0
 Well versed with the subject                              0
 Explains concepts in an understandable way                0
 Use of presentations                                      0
 Degree of difficulty of assignments                       0
 Solves doubts willingly                                   0
 Structuring of the course                                 0
 Provides support for students going above and beyond      0
 Course recommendation based on relevance                  0
 dtype: int64,
 Well versed with the subject                              7.497502
 Explains concepts in an understandable way                6.081918
 Use of presentations                                      5.942058
```

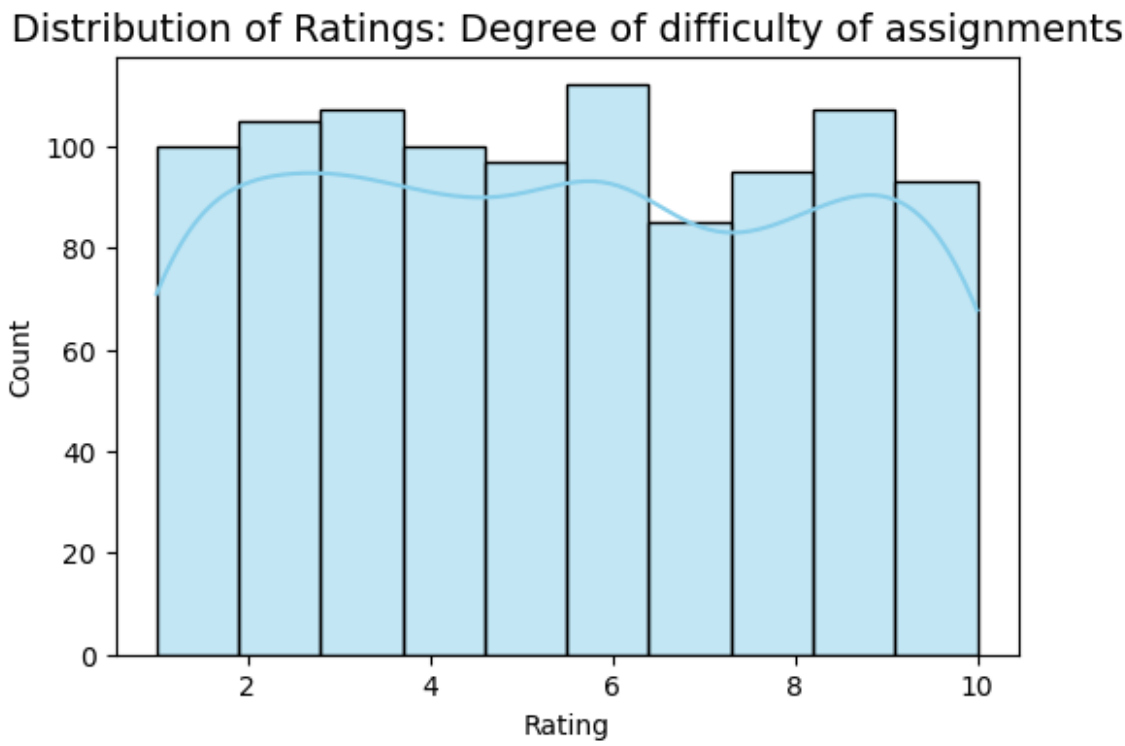
```
Provides support for students going above and beyond    5.662338
Structuring of the course                               5.636364
dtype: float64,
0    5.000
1    4.875
2    4.375
3    5.875
4    7.500
dtype: float64)
```

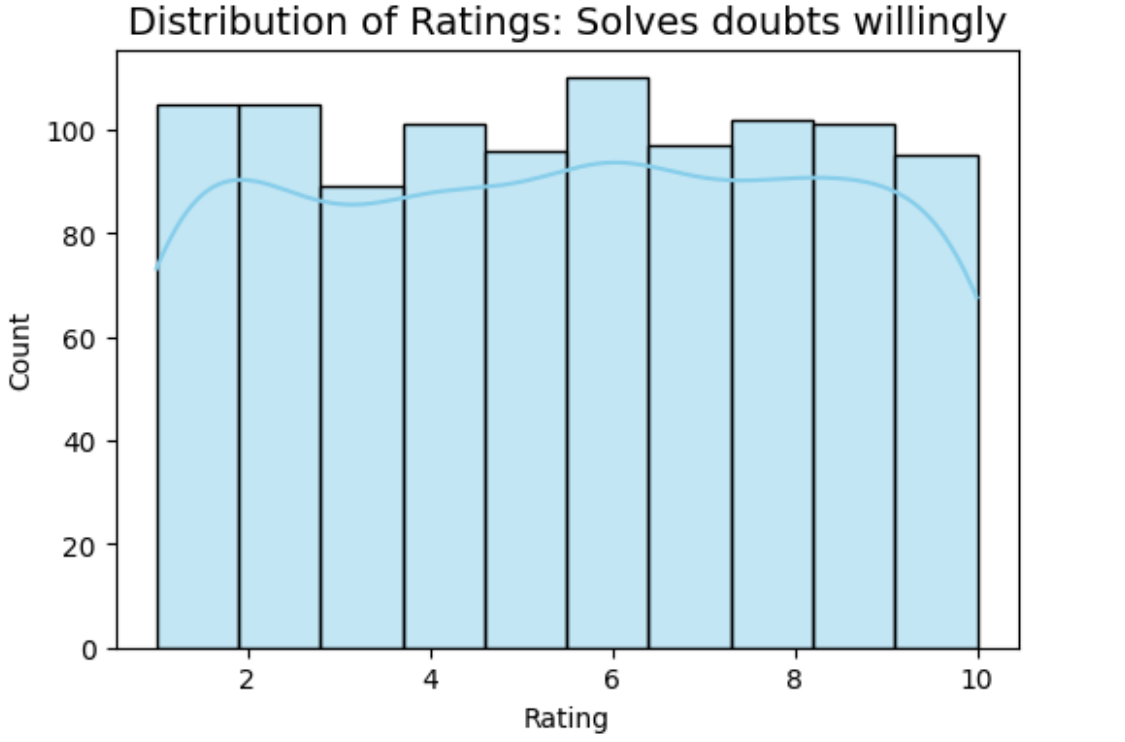
```python
for col in df_clean.drop(columns=['Student ID']).columns:
  plt.figure(figsize=(6,4))
  sns.histplot(df_clean[col], bins=10, kde=True, color='skyblue')
  plt.title(f"Distribution of Ratings: {col}", fontsize=14)
  plt.xlabel("Rating")
  plt.ylabel("Count")
  plt.show()
```

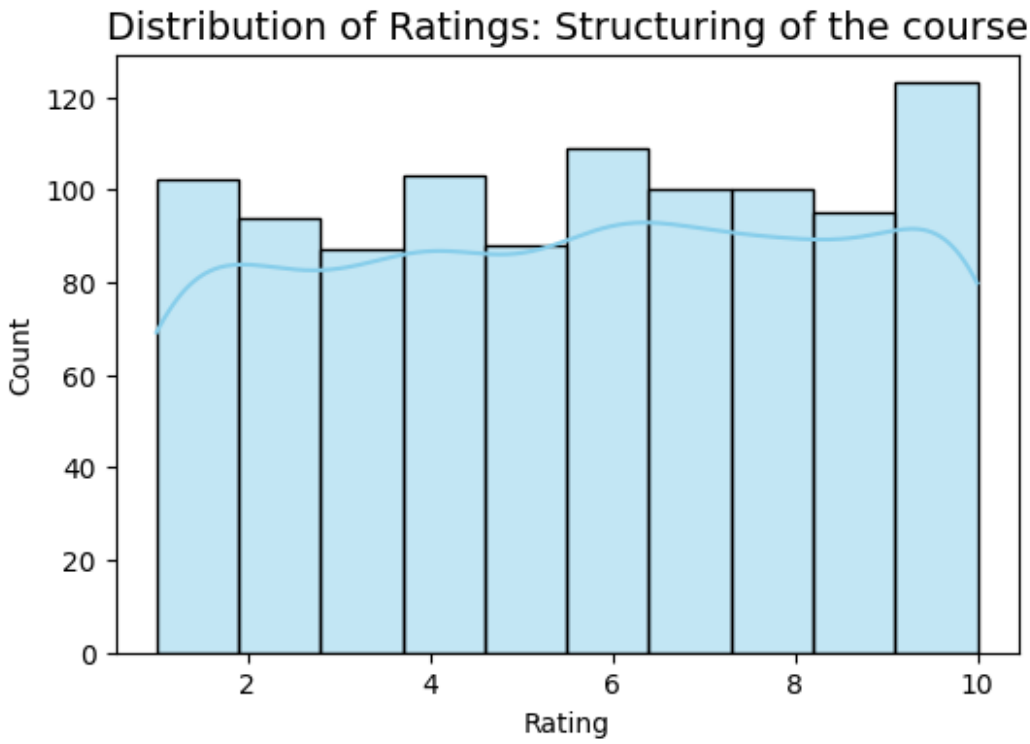


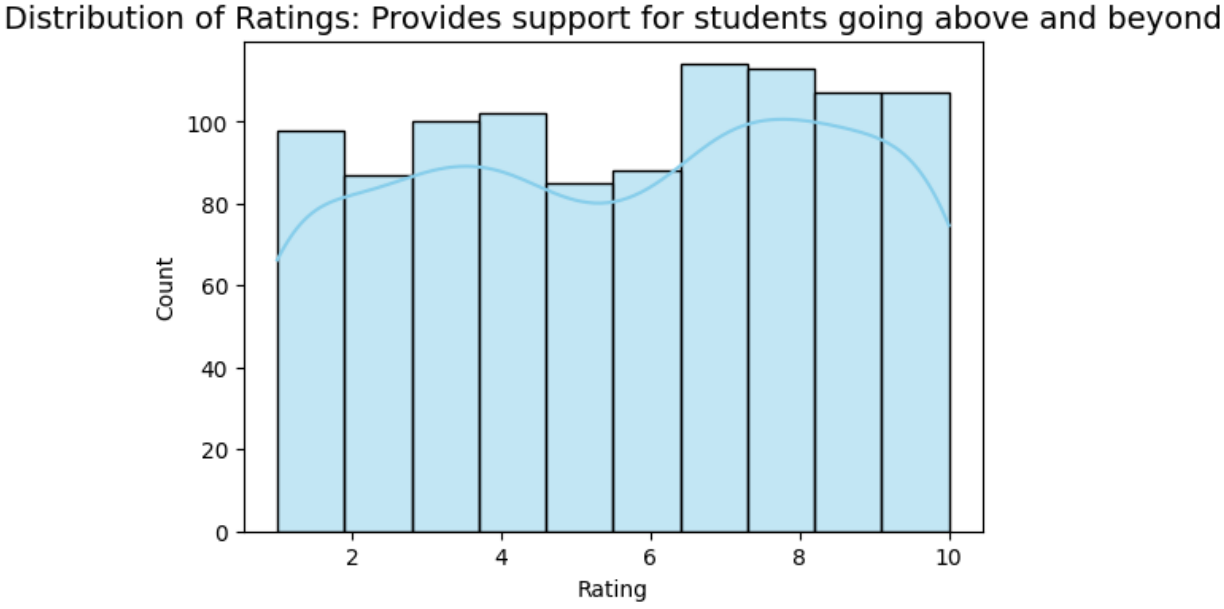

Distribution of Ratings: Explains concepts in an understandable way
Count
Rating
0
20
40
60
80
100
120
2
3
4
5
6
7
8
9
10

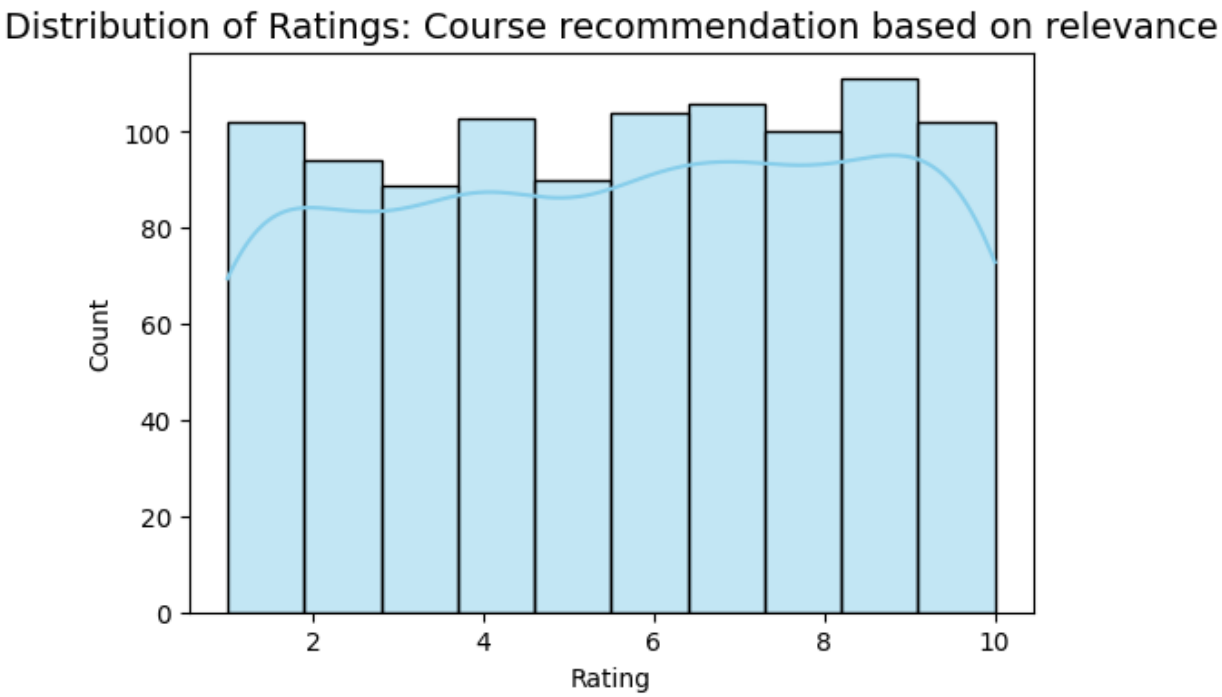

Distribution of Ratings: Use of presentations
Count
Rating
0
50
100
150
200
4.0
4.5
5.0
5.5
6.0
6.5
7.0
7.5
8.0


Distribution of Ratings: Degree of difficulty of assignments
Count
Rating
0
20
40
60
80
100
2
4
6
8
10


Distribution of Ratings: Solves doubts willingly
Count
Rating
0
20
40
60
80
100
2
4
6
8
10


Distribution of Ratings: Structuring of the course
Count
Rating
0
20
40
60
80
100
120
2
4
6
8
10


Distribution of Ratings: Provides support for students going above and beyond
Count
Rating
0
20
40
60
80
100
2
4
6
8
10




```
top_aspects = avg_scores.head(3)
bottom_aspects = avg_scores.tail(3)
top_aspects
bottom_aspects
```

```
Course recommendation based on relevance    5.598402
Solves doubts willingly                     5.474525
Degree of difficulty of assignments         5.430569
dtype: float64
```

```
insights = {
"Top 3 Aspects": top_aspects.to_dict(),
"Bottom 3 Aspects": bottom_aspects.to_dict(),
"Overall Average Satisfaction": round(overall_satisfaction.mean(), 2),
"Highest Rating": round(avg_scores.max(), 2),
"Lowest Rating": round(avg_scores.min(), 2)
}

insights
```

```
{'Top 3 Aspects': {'Well versed with the subject': 7.497502497502498,
  'Explains concepts in an understandable way': 6.081918081918082,
  'Use of presentations': 5.942057942057942},
 'Bottom 3 Aspects': {'Course recommendation based on relevance':
5.5984015984015985,
  'Solves doubts willingly': 5.474525474525475,
  'Degree of difficulty of assignments': 5.430569430569431},
 'Overall Average Satisfaction': np.float64(5.92),
```

```
 'Highest Rating': 7.5,
 'Lowest Rating': 5.43}
```

```
import pandas as pd
import matplotlib.pyplot as plt
from wordcloud import WordCloud, STOPWORDS
from textblob import TextBlob
import nltk

 from nltk.corpus import stopwords

import random
def generate_feedback(score):
  if score >= 8:
    return random.choice([
        "The course was excellent and very well structured.",
        "I really enjoyed the lectures and learned a lot.",
        "Great teaching style and very engaging sessions."
    ])
  elif score >= 5:
    return random.choice([
        "The course was okay but could be improved in some areas.",
        "Some topics were clear, others a bit confusing.",
        "Average experience, assignments were manageable."
    ])
  else:
    return random.choice([
        "The course was difficult to follow and needs improvement.",
        "I struggled to understand the concepts.",
        "Poor explanations and unclear assignments."
    ])

df_clean['Overall_Score'] = df_clean.drop(columns=['Student
ID']).mean(axis=1)

df_clean['Feedback'] =
df_clean['Overall_Score'].apply(generate_feedback)

df_clean.to_csv("student_feedback_with_comments.csv", index=False)

 print(df_clean[['Overall_Score', 'Feedback']].head())
```

```
   Overall_Score                                          Feedback
0          5.000  Average experience, assignments were manageable.
1          4.875           I struggled to understand the concepts.
2          4.375        Poor explanations and unclear assignments.
3          5.875  Average experience, assignments were manageable.
4          7.500   Some topics were clear, others a bit confusing.
```

```
feedback_column = "Feedback"
for col in df.columns:
```

```
    if 'feedback' in col.lower():
        feedback_column = col
        break
if not feedback_column:
    raise ValueError("No column containing 'feedback' found in the
CSV.")

def clean_text(text):
    if pd.isnull(text):
        return ""
    text = str(text).lower()
    text = "".join(char for char in text if char.isalpha() or
char.isspace())
    from nltk.corpus import stopwords
    stop_words = set(stopwords.words('english'))
    text = " ".join(word for word in text.split() if word not in
stop_words)
    return text
feedback_column = "Feedback"
df_clean['Cleaned_Feedback'] =
df_clean[feedback_column].apply(clean_text)
```

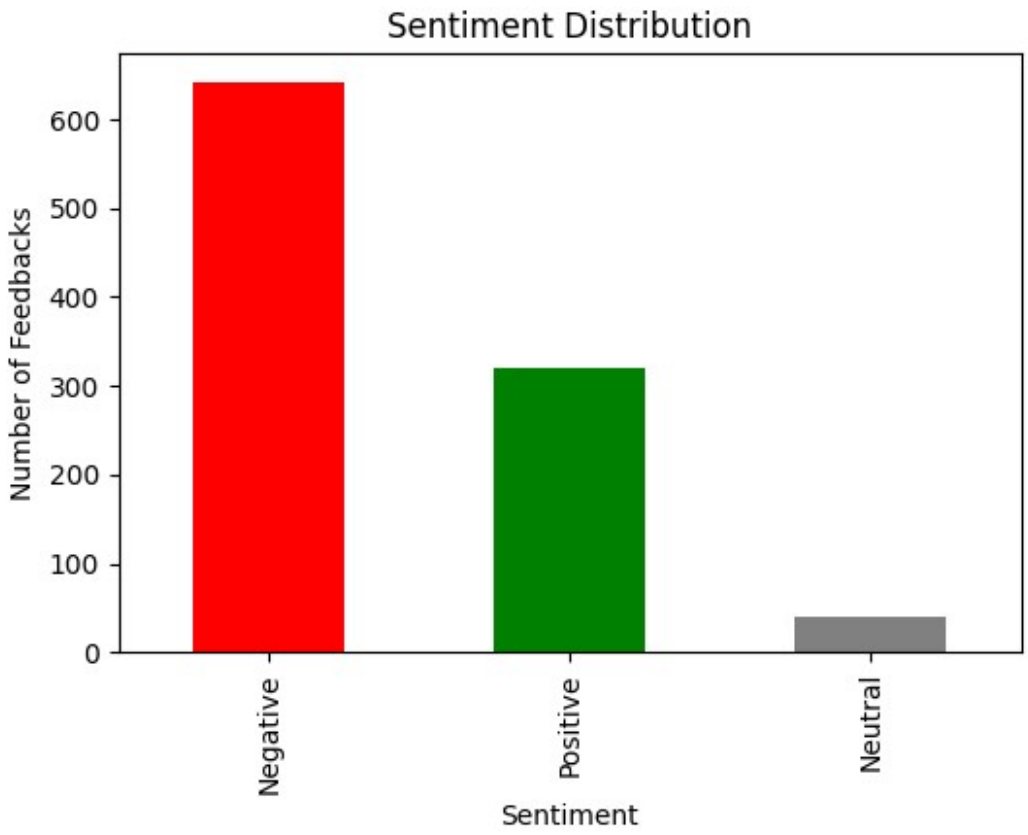
```
---------------------------------------------------------------------
-----
LookupError                               Traceback (most recent call
last)
/usr/local/lib/python3.12/dist-packages/nltk/corpus/util.py in
__load(self)
     83                 try:
---> 84                     root =
nltk.data.find(f"{self.subdir}/{zip_name}")
     85                 except LookupError:

/usr/local/lib/python3.12/dist-packages/nltk/data.py in
find(resource_name, paths)
    578     resource_not_found = f"\n{sep}\n{msg}\n{sep}\n"
--> 579     raise LookupError(resource_not_found)
    580

LookupError:
***********************************************************************
  Resource stopwords not found.
  Please use the NLTK Downloader to obtain the resource:

  >>> import nltk
  >>> nltk.download('stopwords')

  For more information see: https://www.nltk.org/data.html

  Attempted to load corpora/stopwords.zip/stopwords/
```

```
  Searched in:
    - '/root/nltk_data'
    - '/usr/nltk_data'
    - '/usr/share/nltk_data'
    - '/usr/lib/nltk_data'
    - '/usr/share/nltk_data'
    - '/usr/local/share/nltk_data'
    - '/usr/lib/nltk_data'
    - '/usr/local/lib/nltk_data'
**********************************************************************


During handling of the above exception, another exception occurred:

LookupError                               Traceback (most recent call
last)
/tmp/ipython-input-2779567441.py in <cell line: 0>()
      9     return text
     10 feedback_column = "Feedback"
---> 11 df_clean['Cleaned_Feedback'] =
df_clean[feedback_column].apply(clean_text)

/usr/local/lib/python3.12/dist-packages/pandas/core/series.py in
apply(self, func, convert_dtype, args, by_row, **kwargs)
   4922             args=args,
   4923             kwargs=kwargs,
-> 4924         ).apply()
   4925
   4926     def _reindex_indexer(

/usr/local/lib/python3.12/dist-packages/pandas/core/apply.py in
apply(self)
   1425
   1426         # self.func is Callable
-> 1427         return self.apply_standard()
   1428
   1429     def agg(self):

/usr/local/lib/python3.12/dist-packages/pandas/core/apply.py in
apply_standard(self)
   1505         #  Categorical (GH51645).
   1506         action = "ignore" if isinstance(obj.dtype,
CategoricalDtype) else None
-> 1507         mapped = obj._map_values(
   1508             mapper=curried, na_action=action,
convert=self.convert_dtype
   1509         )

/usr/local/lib/python3.12/dist-packages/pandas/core/base.py in
```

```
_map_values(self, mapper, na_action, convert)
    919             return arr.map(mapper, na_action=na_action)
    920
--> 921         return algorithms.map_array(arr, mapper,
na_action=na_action, convert=convert)
    922
    923     @final

/usr/local/lib/python3.12/dist-packages/pandas/core/algorithms.py in
map_array(arr, mapper, na_action, convert)
   1741     values = arr.astype(object, copy=False)
   1742     if na_action is None:
-> 1743         return lib.map_infer(values, mapper, convert=convert)
   1744     else:
   1745         return lib.map_infer_mask(

lib.pyx in pandas._libs.lib.map_infer()

/tmp/ipython-input-2779567441.py in clean_text(text)
      5     text = "".join(char for char in text if char.isalpha() or
char.isspace())
      6     from nltk.corpus import stopwords
----> 7     stop_words = set(stopwords.words('english'))
      8     text = " ".join(word for word in text.split() if word not
in stop_words)
      9     return text

/usr/local/lib/python3.12/dist-packages/nltk/corpus/util.py in
__getattr__(self, attr)
    118             raise AttributeError("LazyCorpusLoader object has
no attribute '__bases__'")
    119
--> 120         self.__load()
    121         # This looks circular, but its not, since __load()
changes our
    122         # __class__ to something new:

/usr/local/lib/python3.12/dist-packages/nltk/corpus/util.py in
__load(self)
     84                     root =
nltk.data.find(f"{self.subdir}/{zip_name}")
     85                 except LookupError:
---> 86                     raise e
     87
     88         # Load the corpus.

/usr/local/lib/python3.12/dist-packages/nltk/corpus/util.py in
__load(self)
     79         else:
     80             try:
```

```
---> 81                     root =
nltk.data.find(f"{self.subdir}/{self.__name}")
     82             except LookupError as e:
     83                 try:

/usr/local/lib/python3.12/dist-packages/nltk/data.py in
find(resource_name, paths)
    577     sep = "*" * 70
    578     resource_not_found = f"\n{sep}\n{msg}\n{sep}\n"
--> 579     raise LookupError(resource_not_found)
    580
    581

LookupError:
**********************************************************************
  Resource stopwords not found.
  Please use the NLTK Downloader to obtain the resource:

  >>> import nltk
  >>> nltk.download('stopwords')

  For more information see: https://www.nltk.org/data.html

  Attempted to load corpora/stopwords

  Searched in:
    - '/root/nltk_data'
    - '/usr/nltk_data'
    - '/usr/share/nltk_data'
    - '/usr/lib/nltk_data'
    - '/usr/share/nltk_data'
    - '/usr/local/share/nltk_data'
    - '/usr/lib/nltk_data'
    - '/usr/local/lib/nltk_data'
**********************************************************************


import nltk
nltk.download('stopwords')

[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.

True

def clean_text(text):
    if pd.isnull(text):
        return ""
    text = str(text).lower()
    text = "".join(char for char in text if char.isalpha() or
char.isspace())
```

```
    from nltk.corpus import stopwords
    stop_words = set(stopwords.words('english'))
    text = " ".join(word for word in text.split() if word not in
stop_words)
    return text
feedback_column = "Feedback"
df_clean['Cleaned_Feedback'] =
df_clean[feedback_column].apply(clean_text)

from textblob import TextBlob
def get_sentiment(text):
    analysis = TextBlob(text)
    polarity = analysis.sentiment.polarity
    if polarity > 0:
        return 'Positive'
    elif polarity < 0:
        return 'Negative'
    else:
        return 'Neutral'
df_clean['Sentiment'] =
df_clean['Cleaned_Feedback'].apply(get_sentiment)
df_clean['Polarity'] = df_clean['Cleaned_Feedback'].apply(lambda x:
TextBlob(x).sentiment.polarity)

import matplotlib.pyplot as plt
plt.figure(figsize=(6,4))
df_clean['Sentiment'].value_counts().plot(kind='bar',
color=['red','green','gray'])
plt.title("Sentiment Distribution")
plt.xlabel("Sentiment")
plt.ylabel("Number of Feedbacks")
plt.show()
```




```
df_clean[[feedback_column, 'Sentiment', 'Polarity']].head(10)
```


```
{"summary":"{\n  \"name\": \"df_clean[[feedback_column, 'Sentiment',
'Polarity']]\",\n  \"rows\": 10,\n  \"fields\": [\n    {\n
\"column\": \"Feedback\",\n      \"properties\": {\n        \"dtype\":
\"string\",\n        \"num_unique_values\": 5,\n        \"samples\":
[\n          \"I struggled to understand the concepts.\",\n
\"The course was okay but could be improved in some areas.\",\n
\"Poor explanations and unclear assignments.\"\n        ],\n
\"semantic_type\": \"\",\n        \"description\": \"\"\n      }\
n    },\n    {\n      \"column\": \"Sentiment\",\n
\"properties\": {\n        \"dtype\": \"category\",\n
\"num_unique_values\": 3,\n        \"samples\": [\n
\"Negative\",\n          \"Neutral\",\n          \"Positive\"\n
],\n        \"semantic_type\": \"\",\n        \"description\": \"\"\n
}\n    },\n    {\n      \"column\": \"Polarity\",\n
\"properties\": {\n        \"dtype\": \"number\",\n        \"std\":
0.3153481321404084,\n        \"min\": -0.4,\n        \"max\": 0.5,\n
\"num_unique_values\": 5,\n        \"samples\": [\n          0.0,\n
0.5,\n          -0.4\n        ],\n        \"semantic_type\": \"\",\n
\"description\": \"\"\n      }\n    }\n  ]\n}","type":"dataframe"}
```


```
import pandas as pd
df = pd.read_csv("student_feedback_with_comments.csv")
```

```
print(df.columns.tolist())
print(df.head())
```

```
['Student ID', 'Well versed with the subject', 'Explains concepts in
an understandable way', 'Use of presentations', 'Degree of difficulty
of assignments', 'Solves doubts willingly', 'Structuring of the
course', 'Provides support for students going above and beyond',
'Course recommendation based on relevance', 'Overall_Score',
'Feedback']
   Student ID  Well versed with the subject  \
0         340                             5
1         253                             6
2         680                             7
3         806                             9
4         632                             8

   Explains concepts in an understandable way  Use of presentations  \
0                                           2                     7
1                                           5                     8
2                                           7                     6
3                                           6                     7
4                                          10                     8

   Degree of difficulty of assignments  Solves doubts willingly  \
0                                    6                        9
1                                    6                        2
2                                    5                        4
3                                    1                        5
4                                    4                        6

   Structuring of the course  \
0                          2
1                          1
2                          2
3                          9
4                          6

   Provides support for students going above and beyond  \
0                                                  1
1                                                  2
2                                                  3
3                                                  4
4                                                  9

   Course recommendation based on relevance  Overall_Score  \
0                                         8          5.000
1                                         9          4.875
2                                         1          4.375
3                                         6          5.875
4                                         9          7.500
```

```
                                              Feedback
0  Average experience, assignments were manageable.
1           I struggled to understand the concepts.
2        Poor explanations and unclear assignments.
3  Average experience, assignments were manageable.
4   Some topics were clear, others a bit confusing.
```

```
df.columns = df.columns.str.strip().str.replace(' ', '_').str.lower()
print(df.columns)
```

```
Index(['student_id', 'well_versed_with_the_subject',
       'explains_concepts_in_an_understandable_way',
'use_of_presentations',
       'degree_of_difficulty_of_assignments',
'solves_doubts_willingly',
       'structuring_of_the_course',
       'provides_support_for_students_going_above_and_beyond',
       'course_recommendation_based_on_relevance', 'overall_score',
       'feedback'],
      dtype='object')
```

```
from textblob import TextBlob
import pandas as pd
df = pd.read_csv("student_feedback_with_comments.csv")
df.dropna(subset=['Feedback'], inplace=True)
df['Feedback'] = df['Feedback'].str.strip()
def analyze_sentiment(text):
    polarity = TextBlob(text).sentiment.polarity
    if polarity > 0:
        sentiment = "Positive"
    elif polarity < 0:
        sentiment = "Negative"
    else:
        sentiment = "Neutral"
    return polarity, sentiment
df['sentiment_polarity'], df['sentiment_category'] =
zip(*df['Feedback'].apply(analyze_sentiment))
df.to_csv("student_feedback_with_sentiment.csv", index=False)
print("Sentiment analysis complete. File saved as
student_feedback_with_sentiment.csv")
```

```
Sentiment analysis complete. File saved as
student_feedback_with_sentiment.csv
```